

SWE 573

SOFTWARE DEVELOPMENT PRACTICE

Project Name: Connect The Dots

Deployment URL: <http://45.67.203.138:3000>

GitHub Repo URL: <https://github.com/sbrsn97/SWE573>

Git Tag Version URL: <https://github.com/sbrsn97/SWE573/releases/tag/v0.9.3>

Course: SWE 573 Software Development Practice, Spring 2025

Author: Sabri ŞEN

Date: 18 May 2025

HONOR CODE

Related to the submission of all the project deliverables for the Swe573 Spring 2025 semester project reported in this report, I Sabri ŞEN declare that:

- I am a student in the Software Engineering MS program at Bogazici University and am registered for Swe573 course during the Spring 2025 semester.
- All the material that I am submitting related to my project (including but not limited to the project repository, the final project report, and supplementary documents) have been exclusively prepared by myself.
- I have prepared this material individually without the assistance of anyone else with the exception of permitted peer assistance which I have explicitly disclosed in this report.

Sabri ŞEN

Signature:

TESTING CREDENTIALS

- **Username:** admin
- **Password:** 123456

THIRD-PARTY SOFTWARE & LICENSES

- React 19 (MIT)
- React DOM 19.0.0 (MIT)
- React Router (MIT)
- PrimeReact 10.9.3 (MIT)
- Tailwind CSS 4.0.14 (MIT)
- React Flow 11.10.4 (MIT)
- Vite 6.2.0 (MIT)
- TypeScript 5.7.2 (MIT)
- ESLint 9.21.0 (MIT)
- Java 17 (GPLv2+CPE)
- Spring Boot (Apache 2.0)
- Spring Security (Apache 2.0)
- Stanford CoreNLP 4.5.5 (GPLv3+)

TABLE OF CONTENTS

1. Overview
2. Software Requirements Specification
3. Design Documents
4. Requirement Status
5. Deployment Status
6. Installation Instructions
7. User Manual
8. Test Results
9. References

1 Overview

System Name: Connect The Dots

Purpose: Connect The Dots is a knowledge management and sharing platform that allows users to create, share, and interact with knowledge graphs through threads. It serves as a collaborative platform for organizing and connecting information in a visual, graph-based format.

Target Users:

- Researchers and academics
- Knowledge workers
- Students
- Anyone interested in creating and sharing knowledge in a structured, visual format

Main Features:

- User Management:
 - User registration and authentication
 - Profile management with customizable information
 - User roles and permissions
 - Following system for users
- Thread Management:
 - Create and share knowledge threads
 - Visual graph editor for creating knowledge connections
 - Tag-based categorization
 - Thread visibility controls
 - Thread history tracking
- Knowledge Graph Features:
 - Interactive graph editor
 - Node and edge creation
 - Relationship visualization
 - Wikidata integration for research
 - Graph arrangement and organization tools
- Social Features:
 - Commenting system
 - Upvoting/downvoting
 - Following threads
 - Notifications for interactions
- Search and Discovery:
 - Advanced search functionality
 - Tag-based filtering
 - Trending content
 - Similar thread recommendations

- Analytics and Insights:
 - Thread popularity metrics
 - Trending content analysis
 - Thread history and version tracking
- Technical Architecture:
 - Frontend:
 - Built with React and TypeScript
 - Uses PrimeReact for UI components
 - TailwindCSS for styling
 - Vite as the build tool
 - Modern authentication system with JWT
 - Backend:
 - Spring Boot 3.2.3
 - Java 17+
 - PostgreSQL database
 - JPA/Hibernate for data persistence
 - RESTful API architecture
 - JWT authentication
- Deployment:
 - Docker containerization
 - PostgreSQL database
 - Support for both development and production environments
- Security Features:
 - JWT-based authentication
 - Role-based access control
 - Secure password handling
 - API endpoint protection
 - CORS configuration
 - Environment variable management for sensitive data
- Notable Technical Features:
 - Notifications
 - Graph visualization
 - Wikidata integration
 - Thread version history
 - User activity tracking
 - Advanced search capabilities
 - Recommendation system
 - Analytics and insights
 - Admin panel
 - Profanity system

The system is designed to be scalable, secure, and user-friendly, with a focus on knowledge sharing and collaboration. It can be deployed either through Docker for easy setup or through local development for customization and testing.

2 Software Requirements Specification

Describe functional and non-functional requirements. You may include/use your SRS document or a tabular

ID	Requirement	Description	Status
FR1	User Registration	Allow new users to register with a unique username, valid email, and password.	✓
FR2	Profile Management	Enable users to update their profile (profession, description, tags, birth date etc...).	✓
FR3	Create Node	Authorized users can create a node with title, description, tags.	✓
FR4	Edit Node	Maintain version history: allow edits to a node's title, description, tags.	✓
FR5	Delete Node	Allow node owners/admins to delete a node.	✓
FR6	Private Node Flag	Node owners or admins can mark a node as private.	✗
FR7	Versioning & History	Track every change (create, edit, delete) with timestamp & user.	✓
FR8	Connect Nodes	Allow users to link two nodes with a labeled relationship.	✓

FR9	Disconnect Nodes	Allow users to remove an existing node-to-node relationship.	✓
FR10	Log Relationship Events	Log each connect/disconnect with timestamp and user ID.	✓
FR11	Wikidata Query	Enable users to fetch summary, aliases, references for a node from Wikidata.	✓
FR12	Display External Data	Show fetched Wikidata data in an organized format on the node detail page.	✓
FR13	Wikidata Error Handling	Gracefully handle API errors or unavailability.	✓
FR14	Authentication Enforcement	Require authentication for all create/edit/delete actions.	✓
FR15	Role-Based Access Control	Enforce visibility: private nodes only to owners & admins; public nodes to all authenticated users.	✗
FR16	Version Revert	Allow owners/admins to revert a node to any prior version.	✗
NFR 1	Performance	API responses < 500 ms under normal load.	✓
NFR 2	Availability	System uptime ≥ 99.9 % over any 30-day period.	✓

NFR 3	Security	Encrypt passwords & sensitive data at rest; use HTTPS for all transport.	✓
NFR 4	Usability	First-time users must be able to create & connect a node without consulting documentation.	✓
NFR 5	Scalability	Architecture must support ≥ 100 concurrent users; allow easy horizontal scaling.	✓
NFR 6	Maintainability	Code follows naming/documentation standards; modular design to onboard new features easily.	✓

3 Design Documents

-

5 Deployment Status

- URL: <http://45.67.203.138:3000>
- Dockerized?: Yes, see <https://github.com/sbrsn97/SWE573/blob/main/docker-compose.yml>
- Environment details: Node v18, Java 17, PostgreSQL 15

6 Full Installation Instructions

Step-by-step guide to get the system up and running, including Docker:

(See: https://github.com/sbrsn97/SWE573/blob/main/docs/DEPLOYMENT_GUIDE.md)

1. Clone repo: `git clone https://github.com/yourusername/SWE573.git`
2. `cd SWE573`
3. Create a `.env` file based on the example: `cp env.example .env`
4. Modify the `.env` file with your configurations:
 - a. `POSTGRES_DB=connect_the_dots_db`
 - b. `POSTGRES_USER=postgres`
 - c. `POSTGRES_PASSWORD=your_secure_password`
 - d. `JWT_SECRET=your_secure_jwt_secret_at_least_32_characters`
5. Build containers: `docker-compose up --build`
6. Start the containers: `docker-compose up -d`
7. Access the application:
 - a. Frontend: <http://localhost:3000>
 - b. Backend API: <http://localhost:8080/api>
 - c. PGAdmin: <http://localhost:5050>

The backend creates an **admin user** with the **username = 'admin'** and **password = '123456'** at the first start. (If the users table is empty on project launch.)

7 User Manual

See: https://github.com/sbrsn97/SWE573/blob/main/docs/USER_GUIDE.md

8 Test Results

8.1 Unit Tests

- Total tests: 200
- Passed: 199
- Coverage: 99%